FROM RAW DATA TO REGRESSION

EDA, Preprocessing & Linear Regression

A One-Shot Foundations Class: from exploring data to building your first ML models

Today's Roadmap

Everything between a raw CSV and a working regression model

- 1. Exploratory Data Analysis (EDA) — understand the data
- 2. Train-Test Split & Missing Value Handling
- 3. Prerequisites of Outlier Handling
- 4. Outlier Detection & Handling
- 5. Encoding categorical variables
- 6. Feature Scaling
- 7. AI/ML Overview + Simple Linear Regression & Gradient Descent
- 8. Multiple Linear Regression
- 9. Multiple Linear Regression with scikit-learn

1. Exploratory Data Analysis (EDA)

Understand before you model

What is it?

- The process of summarizing, visualizing, and questioning a dataset before building any model**Why do we do it?**
- Reveals data types, missing values, and outliers early
- Shows relationships between features and the target
- Wrong assumptions here silently break every step downstream**Typical EDA toolkit**
- Shape, dtypes, describe() — structure & summary statistics
- Histograms / KDE — distribution of a single variable (univariate)
- Scatter plots, box plots, correlation heatmaps — relationships (bivariate)

2. Train-Test Split & Missing Values

Split first — always

Train-Test Split

- Divide data into a training set (model learns) and a test set (model is evaluated on unseen data)
- Typical ratio: 70-80% train, 20-30% test
- Prevents overestimating performance — the model must prove itself on data it never saw

Missing Value Handling

- Real data is rarely complete — sensors fail, forms are skipped
- Options: drop rows/columns, or impute (fill in) values
- Numeric → mean or median (median if outliers exist)
- Categorical → mode (most frequent category)
- Golden rule: compute the fill value from TRAIN only, then apply to both train & test — never let test data leak into training decisions

3. Prerequisites of Outlier Handling

Look before you treat

An outlier isn't automatically "wrong" — always ask:

- Is it a data-entry error, or a real (rare) observation?
- What does the distribution look like? (check histogram / box plot first — most outlier methods assume a roughly symmetric distribution)
- Will removing this point hurt the model's ability to generalize?
- Am I computing bounds on the training set only, to avoid leakage?**Correct order of operations**

Visualize → Decide method → Compute bounds on train → Apply to train & test

4. Outlier Detection & Handling

Finding and treating extreme values

Detection: IQR Method

- Q1 = 25th percentile, Q3 = 75th percentile
- IQR = Q3 − Q1
- Lower bound = Q1 − 1.5 × IQR
- Upper bound = Q3 + 1.5 × IQR
- Anything outside these bounds is flagged as an outlier
- Alternative: Z-score method (flag points beyond ~3 standard deviations)

Handling Strategies

- Remove — drop the rows (only if truly erroneous & few in number)
- Cap / Winsorize — clip values to the lower/upper bound (keeps the row, reduces distortion)
- Transform — apply log/sqrt to shrink the effect of large values
- Bounds are always computed on train, then applied to test too

5. Encoding Categorical Variables

Turning text into numbers models can use

Label Encoding

- Assigns an integer to each category (e.g. male=1, female=0)
- Best for binary categories, or ordinal data with a natural order (e.g. low/medium/high)
- Risk: for non-ordinal data, the model may wrongly assume an order (2 > 1 > 0)

One-Hot Encoding

- Creates a new 0/1 column for each category
- Best for nominal categories with no order (e.g. region, color)
- Drop one column (drop_first) to avoid the 'dummy variable trap' — redundant, perfectly correlated columns

6. Feature Scaling

Putting features on the same footing

Why scale?

- Features often live on very different ranges (age: 18-64 vs children: 0-4)
- Distance- and gradient-based algorithms (linear regression via gradient descent, KNN, SVM) are sensitive to scale
- Without scaling, large-range features dominate learning and slow convergence
- Fit the scaler on train only, then transform both train & test

Common Techniques

- StandardScaler: (x − mean) / std → mean 0, std 1. Good general default
- MinMaxScaler: squashes values into a fixed range, usually [0, 1]
- Tree-based models (Decision Trees, Random Forest) generally don't need scaling

7. AI / ML Overview

Where does linear regression fit in?

- Artificial Intelligence (AI): the broad goal of machines exhibiting intelligent behavior
- Machine Learning (ML): a subset of AI — instead of hand-coded rules, the system learns patterns from data
- Supervised Learning: model learns from labeled examples (input X, correct output y)**Supervised Learning has two main flavors:**
- Regression — predict a continuous number (e.g. insurance charges, house price)
- Classification — predict a category (e.g. spam / not spam)

7. Simple Linear Regression

Fitting the best straight line

$$y = m \cdot x + c$$

- x = input feature, y = predicted target, m = slope, c = intercept
- Goal: find the m and c that make predictions as close as possible to actual values**Cost Function — Mean Squared Error (MSE)**
- MSE = average of (actual − predicted)² across all points
- A lower MSE means a better-fitting line

7. Gradient Descent

How the model actually "learns" m and c

- Start with random values for m and c
- Compute the cost (MSE) — how wrong is the current line?
- Compute the gradient — the slope of the cost function w.r.t. m and c (direction of steepest increase)
- Update: move m and c a small step in the opposite direction of the gradient
- Repeat for many iterations (epochs) until the cost stops decreasing meaningfully**Learning rate**
- Controls the step size — too high overshoots and diverges, too low is painfully slow

8. Multiple Linear Regression

Extending to many features

$$y = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b$$

- Real-world problems rarely depend on just one input — charges depend on age, bmi, smoker status, etc. together
- Each feature gets its own weight (w); the model learns how much each feature contributes
- Same gradient descent idea as simple LR — just extended to vectors/matrices instead of a single number
- This is where scaling matters most: all features must be comparable so no single one unfairly dominates the weights

9. Multiple Linear Regression with scikit-learn

From theory to production-ready code

Why use a library?

- sklearn's LinearRegression solves for the best weights directly (least squares / Normal Equation) — fast, stable, well-tested
- No need to manually pick a learning rate or iterate epochs
- Same underlying math, just optimized and abstracted for you

Evaluating the model

- MSE / RMSE — average squared/root error, penalizes large errors
- MAE — average absolute error, easier to interpret in original units
- R^2 Score — proportion of variance explained (closer to 1 = better fit)

Putting It All Together

Raw Data → EDA → Train/Test Split → Handle Missing Values → Handle Outliers → Encode → Scale → Train Model → Evaluate

Every real ML project — no matter how advanced — follows this same backbone.

Practice next: try this same pipeline on a Kaggle dataset of your choice, and explore Ridge/Lasso regression.